

Paradigmas de Programación

Definición

Son estilos o enfoques fundamentales que guían cómo se diseña y estructura el código para resolver un problema. [1, 2]

Diferentes enfoques, una misma meta...

¡Resolver problemas!

1. Programación Imperativa

Enfoque

Describe el cómo resolver el problema paso a paso mediante una secuencia de instrucciones que cambian el estado del programa.

Características

- ♥ Uso de variables
- ♥ Asignaciones
- ♥ Estructuras de control (bucles y condicionales)

Lenguajes representativos

♥ C, Pascal, Fortran.
[1, 2, 3, 4]

2. Programación Orientada a Objetos (POO)

Enfoque

Organiza el código alrededor de objetos que combinan datos (atributos) y comportamientos (métodos).

Características

- Basado en los pilares fundamentales:
- ♥ Herencia
 - ♥ Polimorfismo
 - ♥ Encapsulación
 - ♥ Abstracción.

Lenguajes representativos

Java, C++, Python.
[1, 2, 3, 4]

3. Programación Funcional

Enfoque

Considera la computación como la evaluación de funciones matemáticas, evitando modificar el estado o los datos mutables.

Características

- ♥ Uso de funciones puras (sin efectos secundarios)
- ♥ Recursión

Lenguajes representativos

Haskell, Lisp, Scala.
[1, 2, 3, 4]

4. Programación Declarativa / Lógica

Enfoque

Describe el qué se quiere obtener en lugar de detallar los pasos. En su variante lógica, define reglas y hechos matemáticos para deducir soluciones.

Características

- ♥ Código más expresivo enfocado estrictamente en metas u objetivos.

Lenguajes representativos

Prolog, SQL.
[1, 2]

class

{ }

f(x)

¡El mismo código, distintos caminos!

SQL

-?-

PARADIGMAS DE PROGRAMACIÓN

Formas de organizar y expresar la solución de problemas mediante programas.

La programación, también es una forma de crear ideas

Código + Lógica = Grandes ideas

IMPERATIVO / PROCEDURAL

Idea principal:

Indica paso a paso qué debe hacer el programa.

Características:

- Variables
- Asignaciones
- Secuencia
- Selección e iteración

Lenguajes representativos:

C, Pascal, Fortran, Ada

ORIENTADO A OBJETOS

Idea principal:

Organiza el programa mediante objetos que reúnen datos y comportamiento.

Características:

- Clases
- Encapsulación
- Herencia
- Polimorfismo

Lenguajes representativos:

Java, C++, C#, Smalltalk, Python

FUNCIONAL

Idea principal:

Expresa el cálculo mediante funciones.

Características:

- Funciones como valores
- Menor estado mutable
- Reducción de efectos secundarios

Lenguajes representativos:

Haskell, Lisp, Scheme, ML, OCaml

LÓGICO / DECLARATIVO

Idea principal:

Describe hechos, reglas o el resultado deseado en lugar de detallar todos los pasos.

Características:

- Declaraciones
- Reglas
- Inferencia

Lenguajes representativos:

Prolog, SQL

DIRIGIDO POR EVENTOS

Idea principal:

El flujo responde a eventos mediante manejadores o listeners.

Características:

- Eventos
- Controladores
- Respuesta a acciones del usuario o sistema

Lenguajes representativos:

JavaScript, C#, Visual Basic

LENGUAJES MULTIPARADIGMA

Permiten combinar varios paradigmas.

Ejemplos:

Python, C++, JavaScript, Scala

¿POR QUÉ SON IMPORTANTES?

- Elegir una estrategia de solución
- Organizar el código
- Facilitar mantenimiento y reutilización
- Adaptar el lenguaje al problema

Los paradigmas no son mutuamente excluyentes

Pequeños pasos, grandes logros